Aura E-Commerce Platform: Comprehensive Project Report

1. Project Overview

Aura is a fully functional, modern e-commerce web application designed with a sleek, minimalist aesthetic. Initially conceived as a Western essential clothing store, the platform has evolved into a global fashion destination featuring a curated **East-Meets-West catalog**—blending affordable daily Indian wear with chic Western essentials and accessories.

2. Technology Stack

The platform is built using a lightweight, fast, and easily deployable stack:

- **Backend:** Python and Flask
- **Frontend:** HTML5, CSS3, and Vanilla JavaScript
- **Data Persistence:** JSON file-based storage (`products.json`, `reviews.json`) and session-based state management.
- **Deployment:** PythonAnywhere (WSGI)

3. Key Features & Capabilities

Dynamic Product Catalog

- **Global Inventory:** Features categories like Sarees, Kurtis, Suits, Dresses, Tops & Coats, Activewear, Accessories, and Makeup.
- **Unique Imagery:** The platform uses 100% unique, AI-generated, high-quality e-commerce photography for every product in the starting catalog.
- **Size Selection:** Customers can select specific sizes (XS through XXL) for clothing items before adding them to the cart.
- **Sales & Promotions:** Includes a dynamic discount system. Customers can apply promo codes (like **AURA20**) at checkout to receive 20% off their entire order. Products also support displaying original crossed-out prices (e.g., ~~₹699~~ ₹450) to highlight deals.

Admin Dashboard & Management

- **Add & Delete Products:** Admins can dynamically add new products (specifying name, price, and category) or delete existing products directly from the web interface.
- **Order Tracking:** Admins can view all incoming orders, what items were purchased, and update the shipping status (Placed, Processing, Shipped, Delivered).
- **Centralized Feedback:** A dedicated section where the admin can read every single product review left by customers across the entire site.

Customer Experience

- **Authentication:** Users can sign up for an account, log in, and manage their session securely.
- **Shopping Cart & Checkout:** A robust JavaScript-powered cart system that calculates subtotals, applies discounts, calculates EMI (Equated Monthly Installments), and processes mock checkouts.
- **Product Reviews:** Customers can leave star ratings and written reviews on individual product pages.

Persistent Data Storage

To ensure data is never lost when the PythonAnywhere server restarts, the application uses local JSON files:

- `products.json`: Saves the entire product catalog, including any items added or deleted by the admin.
- `reviews.json`: Saves all customer feedback and ratings, instantly mapping them to the correct product.

4. Architecture & File Structure

```
womens-shopping-website-python/
│
├── app.py                # Core Flask application and routing logic
├── flask_app.py          # The live copy of app.py used by PythonAnywhere WSGI
├── products.json         # Persistent database for store inventory
├── reviews.json          # Persistent database for customer reviews
│
├── static/
│   ├── css/style.css     # Core aesthetic design and responsive layout
│   ├── js/main.js        # Frontend logic (Cart, Checkout, Promo codes)
│   └── images/           # Unique AI-generated product photography
│
└── templates/            # HTML Templates
    ├── base.html         # Master layout with navigation and footer
    ├── index.html        # Homepage with sale banners and featured items
    ├── shop.html         # Category filtering and product grid
    ├── product.html      # Individual product page with reviews and sizes
    ├── cart.html         # Shopping cart overview
    ├── checkout.html     # Multi-step checkout flow
    ├── admin.html        # The powerful Admin management dashboard
    └── login.html / signup.html / admin_login.html
```

5. Deployment Guide (PythonAnywhere)

The application is designed to be easily updated and deployed on PythonAnywhere via a packaged zip file.

Standard Update Protocol:

1. Package the local directory into `aura_update.zip`.
2. Upload `aura_update.zip` to the **Files** tab on PythonAnywhere.
3. Open the **Bash console**.
4. Run `unzip -o aura_update.zip` to extract the new files.
5. Run `cp app.py flask_app.py` to ensure the WSGI server sees the latest code changes.

6. Navigate to the **Web** tab and click **Reload**.

6. Future Improvements

While the platform is highly capable, future scaling could include:

- Migrating from JSON file storage to a relational database like SQLite or PostgreSQL for handling thousands of concurrent users.
- Implementing a real payment gateway (like Stripe or Razorpay) in place of the mock checkout.
- Adding an image upload feature in the admin panel so admins can upload their own product photos rather than relying on predefined category images.